

1_클래스로더 서브 시스템

`.class` 파일이 디스크에 있다고 저절로 실행되는 것은 아니다. 그 파일을 찾아서 JVM 메모리 영역으로 끌어올리고, 안전한지 검사하고, 실행 가능한 상태로 준비하는 과정이 필요하다. 이 일을 담당하는 것이 **클래스 로더 서브시스템(Class Loader Subsystem)** 이다.

1. 로딩 (Loading)

클래스의 이진 표현을 찾아서 읽어 들이는 단계다.

클래스 로더가 클래스의 정규화된 이름(fully-qualified name)으로 해당 바이트코드를 찾아오고, 이를 바탕으로 **메서드 영역에 클래스 메타데이터**를 만든다. 그리고 이 클래스를 대표하는 **Class 객체를 Heap에 생성**한다.

- 로딩 시점에는 아직 검증이나 값 할당이 이뤄지지 않는다.
- 그냥 데이터를 읽어와 메모리 형태를 갖춰 놓은 상태다.

2. 링킹 (Linking)

로드된 클래스를 JVM의 런타임 상태에 통합해, 실행할 수 있게 만드는 과정이다. 세 개의 하위 단계로 나뉜다.

2.1 검증 (Verification)

로드된 바이트코드가 구조적으로 올바른지, JVM의 제약을 위반하지 않는지 확인한다. 예를 들면 이런 것들이다.

- 스택이 overflow 또는 underflow 하지 않는지
- 타입이 맞는지
- 잘못된 방식으로 JVM의 제약을 위반하지 않는지

이 단계는 **신뢰할 수 없는 출처의 코드가 JVM을 손상시키는 것을 막는다.**

2.2 준비 (Preparation)

클래스의 **static** 변수에 메모리를 할당하고, **기본값으로 초기화**하는 단계다.

여기서 들어가는 것은 개발자가 지정한 실제 값이 아니라, 타입의 기본값이다.

```
static int count = 10;
// 이렇게 작성해도 준비 단계에서는 0이 들어간다.
// 개발자가 지정한 값(10)은 나중에 '초기화' 단계에서 할당된다.
```

2.3 해석 (Resolution)

상수 풀(constant pool) 안의 **심볼릭 참조를 실제 참조로 교체**한다.

- 클래스 파일에는 다른 클래스나 메서드, 필드가 처음엔 이름으로만 박혀 있다.
- 그 이름을 따라가, 실제 메모리상의 위치나 대상으로 연결한다.
- 상황에 따라 그 참조가 실제로 사용되는 시점까지 미뤄질 수 있다. → **lazy resolution**

3. 초기화 (Initialization)

클래스 준비의 마지막 단계다. **static** 변수에 **실제 값을 할당**하고, **static** 초기화 블록을 실행한다. 즉, 여기서 개발자가 지정했던 값으로 초기화된다.

- 클래스는 **필요할 때까지 초기화되지 않는다**. 이것을 지연 초기화(lazy initialization) 원칙이라고 한다.

4. 계층적 클래스 로더와 부모 위임

그렇다면 누가 로딩을 하는가? 클래스 로더는 계층을 이룬다. 최상위인 **부트스트랩 클래스 로더**는 **java.lang.*** 같은 자바 핵심 API를 로드한다.

```

부트스트랩 클래스 로더
├── 플랫폼 클래스 로더
│   └── 애플리케이션(시스템) 클래스 로더 ← 개발자가 작성한 애플리케이션 클래스를 로드

```

핵심은 **부모 위임 모델(Parent Delegation Model)** 이다.

- 어떤 클래스를 로드해달라는 요청이 오면, 로더는 그것을 직접 로드하지 않고 **먼저 부모 로더에게 위임**한다.
- 부모가 로드할 수 있으면 부모가 로드하고, 부모가 못 하면 그제야 자기가 로드한다.

왜 이렇게까지 할까?

- 누군가 악의적으로 가짜 **java.lang.String** 을 만들어도, 그 요청이 부트스트랩 로더에게 먼저 올라가 진짜 표준 **String** 이 로드된다. 위조 클래스가 끼어들 수 없다.
- 같은 클래스가 여러 번 로드되는 것을 막아 일관성을 유지한다.

마무리

```

.class 로딩 (읽어오기)
→ 링크 (검증 · 준비 · 해석)   ※ '준비'에서는 기본값만 들어간다
→ 초기화 (실제 값 할당 + static 블록 실행)   ※ 여기서 실제 값이 들어간다

```

그리고 이 모든 과정에서 클래스 로더는 **부모 위임 모델**로 동작해, 보안과 중복 방지를 보장한다.